

ZERO ONE HACK 2026 — OMIM PROJECT MASTER CONTEXT

Hackathon Context

Zero One Hack 2026 is a 36-hour AI engineering hackathon in Vienna, Austria.

Theme:

"36 hours of real model training, not prompt engineering."

Judging heavily favors:

- working systems
- real infrastructure
- actual AI engineering
- deterministic execution
- deployment
- operational reliability
- model training
- industrial applicability

Judges explicitly prefer:

"A small model trained and demonstrated successfully is better than a giant model that partially works."

Therefore the project should prioritize:

- execution
- architecture
- validation
- reliability
- demonstrable functionality

over:

- hype
- AGI claims
- oversized models
- unfinished research

Available Resources

Featherless AI

Provided during the hackathon.

Capabilities:

- OpenAI-compatible API
- DeepSeek models
- Qwen models
- Kimi models
- Large language models
- Semantic reasoning
- Structured outputs

Recommended Use:

- semantic annotation
- manufacturing explanations
- dataset caption generation
- design intent descriptions
- manufacturing documentation

NOT recommended for:

- authoritative manufacturing validation
- geometry parsing
- graph generation
- deterministic rule execution

LLMs should remain advisory.

Leonardo HPC Infrastructure

Provided through AI Factory Austria.

Infrastructure:

- NVIDIA A100 GPUs
- High-performance compute cluster
- Distributed training capability

Recommended Use:

Future phases:

- Graph Neural Networks
- Manufacturing embeddings
- Manufacturability prediction
- Feature classification
- Large-scale synthetic dataset processing

Not required for MVP.

Project Name

OMIM

Open Manufacturing Intelligence Middleware

Executive Summary

OMIM is NOT:

- CAM software
- CNC controller
- G-code generator
- robotics software
- autonomous machining
- factory automation
- AGI manufacturing

OMIM IS:

A deterministic manufacturing intelligence infrastructure.

The objective is to convert manufacturing geometry into structured manufacturing knowledge.

The project creates:

- Manufacturing Geometry Graphs (MGG)
- manufacturability validation
- semantic manufacturing understanding
- provenance-aware datasets
- synthetic manufacturing datasets

The project is focused on building the infrastructure layer that future manufacturing AI systems can learn from.

Core Problem

Most manufacturing AI systems today operate as:

Geometry ↓ Black Box Model ↓ Output

Problems:

- weak manufacturing grounding
- no provenance
- hallucinations
- poor explainability
- no deterministic validation
- lack of manufacturing datasets

Most systems understand geometry.

They do not understand manufacturing meaning.

OMIM attempts to solve that problem.

Core Philosophy

Hierarchy of truth:

Geometry Truth

Validation Truth

Semantic Truth

AI Truth

Validation always remains authoritative.

AI never overrides geometry.

AI never overrides validation.

Current Scope

Domain:

- cabinetry manufacturing
- modular furniture
- panel manufacturing
- MDF
- plywood
- melamine

Geometry:

- DXF
- 2D
- 2.5D

Manufacturing Features:

- holes
- pockets
- contours
- slots
- drilling
- routing

High-Level Architecture

DXF ↓ Geometry Parser ↓ Canonical Geometry ↓ Manufacturing Geometry Graph (MGG) ↓ Validation Engine ↓ Semantic Annotation ↓ Dataset Export

Everything revolves around the MGG.

Manufacturing Geometry Graph (MGG)

Purpose:

Convert raw geometry into structured manufacturing relationships.

The graph may contain:

Nodes:

- Panel
- Hole
- Pocket
- Slot
- Contour
- Tool
- Operation
- Constraint

Relationships:

- BELONGS_TO
- REQUIRES_TOOL
- ADJACENT_TO
- INTERSECTS
- VIOLATES_CONSTRAINT
- GENERATED_BY
- DERIVED_FROM

The graph should remain minimal and practical.

Avoid ontology bloat.

Validation Engine

Validation is deterministic.

Examples:

- edge clearance
- tool diameter compatibility
- slot width
- contour closure
- pocket accessibility

Validation determines manufacturability.

LLMs cannot override validation.

Semantic Annotation Layer

Purpose:

Add manufacturing meaning.

Example:

Geometry: Circle

Semantic Meaning: Shelf Pin Hole

Geometry: 35mm Hole

Semantic Meaning: Hinge Cup Hole

Geometry: Pocket

Semantic Meaning: Hardware Pocket

Semantics are advisory.

Validation remains authoritative.

Featherless Integration

Featherless should only be used AFTER MGG creation.

Architecture:

MGG ↓ Featherless ↓ Semantic Annotation

Possible tasks:

- manufacturing intent
- feature explanation
- dataset captions
- natural language descriptions
- documentation generation

Example Output:

"This panel appears to be a cabinet side panel containing shelf pin holes and concealed hinge mounting locations."

The LLM explains.

The validator decides.

Provenance System

Every inference should store:

- source
- derivation
- confidence
- evidence

Nothing should appear magically.

MVP (Hackathon Deliverable)

The MVP is:

DXF ↓ Parser ↓ MGG ↓ Validation ↓ Visualization ↓ LLM Semantic Explanation

Input:

cabinet_panel.dxf

Outputs:

graph.json validation.json semantic.json

Optional:

human-readable explanation

This is sufficient for the hackathon.

Graph Visualization

Recommended:

Frontend:

- React
- Cytoscape.js

Purpose:

Display MGG visually.

This is a key demo component.

Users should see:

- holes
- pockets
- contours
- relationships
- constraints

as graph structures.

Dataset Export

Export format:

JSON

Include:

- geometry
- graph
- validation results
- semantic labels
- provenance

The export format should be suitable for future ML training.

Future Phase 2

Synthetic Manufacturing Dataset Generation

Pipeline:

Manufacturing Standards ↓ Rule-Based Generator ↓ Manufacturing Features ↓ Validation ↓ MGG ↓ Dataset Export

Goal:

Generate manufacturing-valid synthetic examples.

Not random CAD.

Grounded manufacturing data.

Future Phase 3

Graph Learning

Pipeline:

MGG Dataset ↓ Graph Neural Network ↓ Manufacturing Embeddings ↓ Prediction Tasks

Potential Tasks:

- manufacturability prediction
- feature classification
- tooling recommendation
- operation prediction

Recommended:

PyTorch Geometric

This phase justifies HPC usage.

Recommended Tech Stack

Backend:

- Python
- FastAPI

Geometry:

- ezdxf
- shapely

Graphs:

- NetworkX

Frontend:

- React
- Cytoscape.js

LLM:

- Featherless API

Future ML:

- PyTorch
- PyTorch Geometric

Deployment:

- Docker

Optional:

- Kubernetes

Guidance For Claude

When generating code:

1. Respect architecture boundaries.
2. Keep validation deterministic.
3. Do not allow LLMs to become authoritative.
4. Prefer simple designs.

5. Prefer explicit schemas.
6. Preserve provenance.
7. Avoid overengineering.
8. Avoid ontology complexity.
9. Focus on the MVP first.
10. Prioritize working demonstrations over ambitious unfinished systems.

The primary objective is a working Manufacturing Geometry Graph pipeline, not a manufacturing AGI.